Mini Project-1 Report

Name: Md Jahidul Islam Noor

Milestone 1:

Which function takes most total time?

-> `mandelbrot_naive` function is calling another function `mandelbrot_point` which is taking the most time 0.826

Are there functions called surprisingly many times?

-> Yes, the built in `abs` function is called many times (part of naive implementation and mandelbrot_point function)

How does the NumPy profile compare to naive?

-> the numpy profile is faster than the naive profile. The naive profile shows that this function is taking 1.322 seconds, but the numpy one is taking 0.106 seconds. Also, there are fewer function calls in the numpy approach.

Where does NumPy spend its time?

-> The time spended by NumPy at any place is very low, but NumPy using its most time on “method 'copy' of 'numpy.ndarray' objects”. It might be using most of his time here because it has to create the numpy array here for the time to do the calculations.

Function level profile of naive approach:

```
Thu Feb 26 14:21:18 2026    naive_profile.prof

         5960971 function calls in 1.322 seconds

   Ordered by: cumulative time
   List reduced from 20 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    1.322    1.322 {built-in method builtins.exec}
        1    0.000    0.000    1.322    1.322 <string>:1(<module>)
        1    0.256    0.256    1.321    1.321 /home/jahid/Aalborg University/second semester/nsc-course/mandelbrot - nsc/mandelbrot_naive.py:22(mandelbrot_naive)
   262144    0.826    0.000    1.065    0.000 /home/jahid/Aalborg University/second semester/nsc-course/mandelbrot - nsc/mandelbrot_naive.py:13(mandelbrot_point)
  5698796    0.240    0.000    0.240    0.000 {built-in method builtins.abs}
        2    0.000    0.000    0.000    0.000 /home/jahid/anaconda3/envs/nsc2026/lib/python3.11/site-packages/numpy/_core/function_base.py:26(linspace)
        1    0.000    0.000    0.000    0.000 {built-in method numpy.zeros}
        2    0.000    0.000    0.000    0.000 {built-in method numpy.arange}
        2    0.000    0.000    0.000    0.000 {method 'reshape' of 'numpy.ndarray' objects}
        2    0.000    0.000    0.000    0.000 {method 'result_type' of 'numpy._core._multiarray_umath._array_converter' objects}
```

Function level profile of the numpy approach:

```
Thu Feb 26 14:21:18 2026    numpy_profile.prof

         77 function calls in 0.106 seconds

   Ordered by: cumulative time
   List reduced from 42 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.106    0.106 {built-in method builtins.exec}
        1    0.001    0.001    0.106    0.106 <string>:1(<module>)
        1    0.103    0.103    0.106    0.106 /home/jahid/Aalborg University/second semester/nsc-course/mandelbrot - nsc/mandelbrot_numpy.py:8(mandelbrot_numpy)
        1    0.000    0.000    0.001    0.001 /home/jahid/anaconda3/envs/nsc2026/lib/python3.11/site-packages/numpy/lib/_function_base_impl.py:5134(meshgrid)
        3    0.000    0.000    0.001    0.000 /home/jahid/anaconda3/envs/nsc2026/lib/python3.11/site-packages/numpy/lib/_function_base_impl.py:5280(<genexpr>)
        2    0.001    0.001    0.001    0.001 {method 'copy' of 'numpy.ndarray' objects}
        1    0.001    0.001    0.001    0.001 /home/jahid/anaconda3/envs/nsc2026/lib/python3.11/site-packages/numpy/_core/numeric.py:98(zeros_like)
        2    0.000    0.000    0.000    0.000 /home/jahid/anaconda3/envs/nsc2026/lib/python3.11/site-packages/numpy/_core/function_base.py:26(linspace)
        1    0.000    0.000    0.000    0.000 /home/jahid/anaconda3/envs/nsc2026/lib/python3.11/site-packages/numpy/lib/_stride_tricks_impl.py:481(broadcast_arrays)
        1    0.000    0.000    0.000    0.000 /home/jahid/anaconda3/envs/nsc2026/lib/python3.11/site-packages/numpy/lib/_stride_tricks_impl.py:546(<listcomp>)
```

Summary Table:

Implementation	Total Runtime (s)	Total Function Calls	Main Time Consumer	Calls to Main Function	Key Observation
Naive	**1.322 s**	**5,960,971**	`mandelbrot_point`	**262,144**	Heavy Python loops cause millions of function calls
NumPy	**0.106 s**	**77**	`mandelbrot_numpy`	**1**	Vectorization removes Python loops

Milestone 2:

cProfile on naive vs NumPy: How many functions appear in each profile? What does this difference tell you about where the work actually happens?
-> For the naive approach, there are 20 functions, and for the numpy approach, there are 42 functions. Even though numpy has more functions than naive, numpy is still faster. The reason numpy uses a lot of helper functions is for better optimization. All these helper functions are written in C, which makes it faster. Numpy takes the work from Python and let C functions do the job. On the other hand, the naive approach is a simple version that calls the same functions multiple times to do the job, and none of these functions are written in C code for optimization. So, from this, it can be said that for numpy, most of the work happens in C code and for naive, most of the work happens in python loop.

line profiler on naive: Which lines dominate runtime? What fraction of total time is spent in the inner loop?
->

```
Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    21                                           @profile
    22                                           def mandelbrot_naive(
    23                                               xmin=-2.0, xmax=1.0,
    24                                               ymin=-1.5, ymax=1.5,    width=1024, height=1024,
    25                                               max_iter=100
    26                                           ):
    27         1        115.8    115.8      0.0      x_vals = np.linspace(xmin, xmax, width)
    28         1         20.5     20.5      0.0      y_vals = np.linspace(ymin, ymax, height)
    29
    30         1         22.6     22.6      0.0      result = np.zeros((height, width), dtype=int)
    31
    32      1025        131.1      0.1      0.0      for i in range(height):
    33   1049600     147160.5      0.1      1.6          for k in range(width):
    34   1048576    1028166.5      1.0     10.9              c = x_vals[k] + 1j * y_vals[i]
    35   1048576    8273910.1      7.9     87.6              result[i, k] = mandelbrot_point(c, max_iter)
    36
    37         1          0.1      0.1      0.0      return result
```

From the above line profile for naive approach, it can be seen that at line 35, the mandelbrot_point function call dominates the runtime. In this naive approach, there are three loops, out them two are inner loops. The most inner loop is in the “mandelbrot_point” which takes 87.6% of the total time.

Based on your profiling results: why is NumPy faster than naive Python?
-> Numpy uses matrix multiplication, where for the naive version it uses loops (many loops, for long iterations). Also the numpy’s code is highly optimized it is written in C, when python calls numpy, python just hand over the task to some highly optimized C functions. But in the naive approach this does not happen.

What would you need to change to make the naive version faster? (hint: what does line profiler tell you about the inner loop?)
-> To make the naive version faster I have to change the mandler_point function, because the bottleneck is there. Lot of the computational time and function calls happened here. Like this function was called 1048576 times and uses 87.6% of the total time. In the inner loop there is abs(z) builtin function call if we use squared magnitude (z.real*z.real + z.imag*z.imag) instead the code might be faster. Also using the JIT or numpy will increase the speed but it will no longer remain naive anymore.

Milestone 3:

Benchmark table of all implemented versions:

Implementation	Runtime (seconds)	Speedup vs Naive
Naive	**3.169**	**1.00×**
NumPy	**0.533**	**5.94×**
Hybrid	**1.073**	**2.95×**
Numba	**0.045**	**70.25×**

Specifications:
- CPU: 13th Gen Intel® Core™ i5-1334U × 12
- Resolution: 1024×1024
- Precision: float64

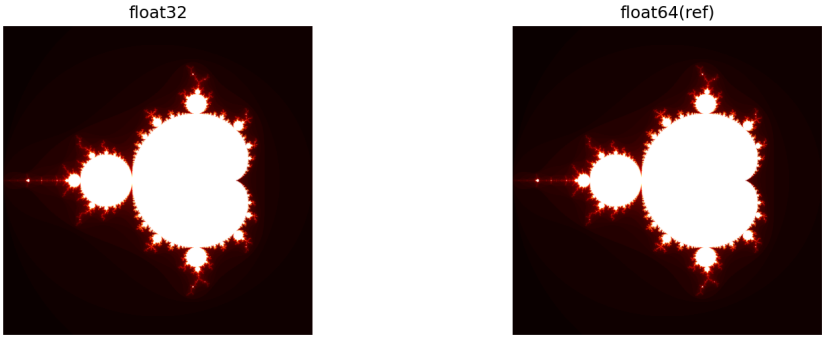
Milestone 4:

Your measured runtimes for float32, float64

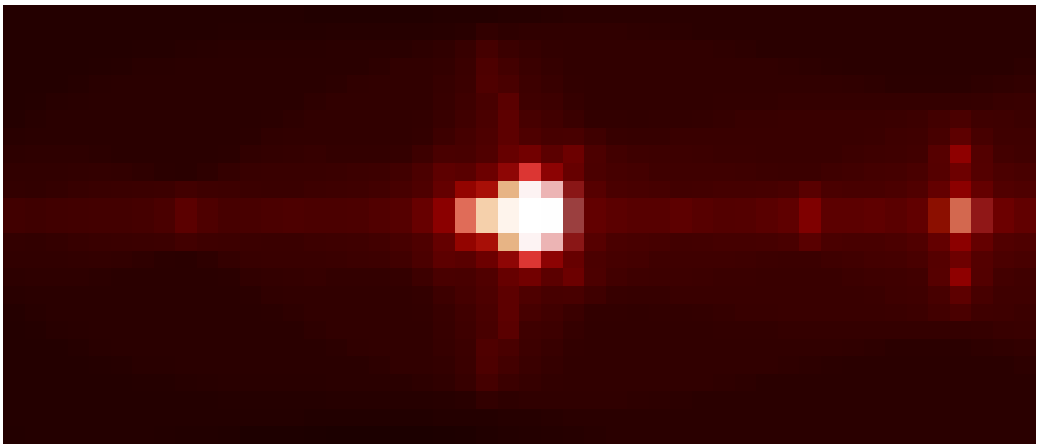
-> For float32 I got 0.467s and for float64 I got 0.142s. Before the experiment, I thought float32 would be faster.

Side-by-side images

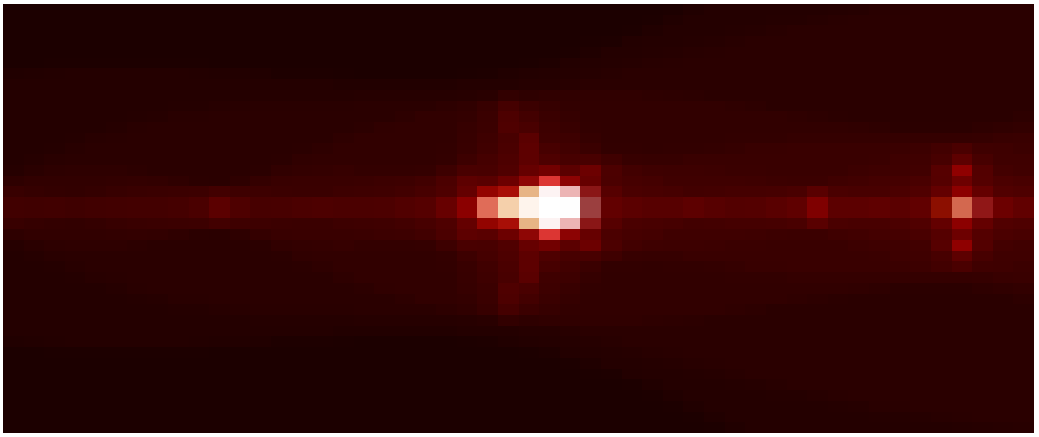
->



Zoomed in float32:



Zoomed in float64:



A brief recommendation: which precision would you use and why?
-> I would choose precision float64. The reason behind is its faster runtime. When njit is used on float32 and flaot64, float64 precision gives faster output. Even though in theory float32 should have given faster result. The reason might be that njit is optimized on float64 not on float32. If we don't use njit both of the precision would be slow, in that case float32 might be faster. For the visual appearance I don't see any difference. So, my choice of choosing float64 is solely based on speed.